High-Level Synthesis of Hardware Accelerators using Bambu

Processor Design
2nd Report

Alba Soldevilla González

April 20, 2026

UNIVERSITAT POLITÈCNICA
DE CATALUNYA
BARCELONATECH

Contents

1 Project Topic and Objectives

For this project, I have been assigned the task of converting software into hardware designs using **High-Level Synthesis (HLS)** techniques and different hardware design tools. The main goal is to translate C/C++ code into synthesizable Hardware Description Language (HDL) implementations such as VHDL and Verilog.

Once the conversion is completed, the generated hardware designs will be synthesized for **Field Programmable Gate Arrays (FPGAs)**. The resulting implementations will then be evaluated using a set of benchmarks in order to analyze their efficiency.

The overall workflow of the project will be as follows:

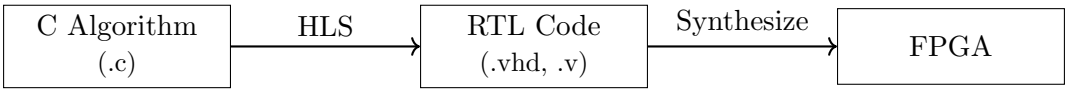

Figure 1: Simplified workflow of the hardware generation process.

2 Tools

To perform the High-Level Synthesis process, two different tools will be used: **Bambu** and MATLAB. While my teammate will focus on MATLAB-based exploration, this work focuses on the use of Bambu for hardware generation.

The generated hardware designs are then synthesized and implemented using **AMD Vivado**, a design suite for AMD FPGAs that covers the full implementation flow: synthesis, placement, routing, and verification.

The workflow presented in Figure 1 can be updated as shown in Figure 2.

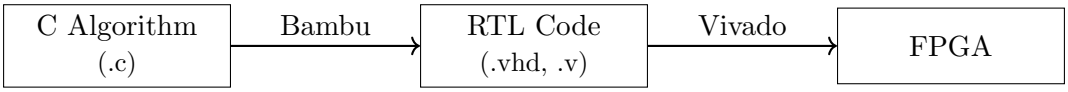

Figure 2: Toolchain used in this project.

At this stage, the focus is on learning how to use Vivado and beginning to apply the full HLS workflow to a set of real benchmarks.

All experiments were carried out on Kubuntu 24.04 LTS (Intel Core i7-1365U) using **PandA Bambu 2024.10** and **Vivado v2025.2**, targeting the `xc7z020clg484-1` (Zynq-7020) device, which is the default generated by Bambu's TCL script.

3 Example: Simple Adder

Before testing any benchmark, the adder example from the previous report is going to be completed by finishing the Synthesize stage.

Find here below the C code of the adder example:

Listing 1: `sum` function in C used as input for the HLS process.

```
int sum(int a, int b){
    return a + b;
}
```

3.1 Hardware Generation

The design was synthesized using the following command:

```
$ bambu sum.c --top-fname=sum
```

The option `--top-fname=sum` was used because the file `sum.c` does not contain a `main()` function but only the function itself. Therefore, Bambu needs to know explicitly which function should be used as the top-level module for hardware generation.

```
root@1b77d6afd9d8:/workspace# ls
HLS_output  build  sum.c  sum.v  synthesize_Synthesis_sum.sh
root@1b77d6afd9d8:/workspace# ls HLS_output/Synthesis/vivado_flow/
input  output  sum.sdc  vivado.tcl
```

Figure 3: Files generated from the `sum.c` source code.

As shown in Figure 3, the RTL file (`sum.v`) was successfully generated, as expected.

3.2 Testbench Simulation

As stated in the Bambu tutorial, the following command can be used to run a simulation:

```
$ bambu sum.c --top-fname=sum --simulate --simulator=VERILATOR
```

Since no `test.xml` file was provided, Bambu automatically generated one with random input parameters. A short simulation report was printed in the terminal and a `results.txt` file was generated in the working directory.

```
Warning: XML file "test.xml" cannot be opened, creating a stub with random values
  Total cycles             : 1 cycles
  Number of executions     : 1
  Average execution        : 1 cycles
root@8c8c3c3f1e59:/workspace# ls
HLS_output           build        simulate_sum.sh  sum.v                          test.xml
bambu_results_0.xml  results.txt  sum.c            synthesize_Synthesis_sum.sh
root@8c8c3c3f1e59:/workspace# cat test.xml
<?xml version="1.0"?>
<function>
  <testbench a="9" b="3"/>
</function>
root@8c8c3c3f1e59:/workspace# cat results.txt
7|9,
0
```

Figure 4: Simulation report of the `sum` function.

The `results.txt` file shown in Figure 4 indicates that no error occurred during the simulation. For the test defined in `test.xml`, using the values $\mathbf{a} = \mathbf{9}$ and $\mathbf{b} = \mathbf{3}$, both the C model (`sum.c`) and the generated RTL implementation (`sum.v`) produced the same result. Therefore, no mismatch was detected between the software model and the synthesized hardware.

3.3 Synthesize with Vivado

As shown in Figures 4 and 3, once the synthesis is complete, Bambu automatically generates an executable script called `synthesize_Synthesis_sum.sh`.

Listing 2 shows its content.

Listing 2: `synthesize_Synthesis_sum.sh` content

```
#!/bin/bash
# Synthesis script for COMPONENT: sum
SWD="$(dirname $(readlink -e $0))"

# STEP: vivado_flow
cd $SWD
ulimit -s 131072; vivado -mode batch -nojournal -nolog -source
    HLS_output/Synthesis/vivado_flow/vivado.tcl
```

Bambu provides a ready-to-use script that launches Vivado in batch mode using an auto-generated `.tcl` file, which handles the full implementation flow: project configuration, Verilog import, timing constraints, and routing. It can be executed as follows:

```
$ bash syntehsize_Synthesis_sum.sh
```

Once executed, Vivado performs the full implementation flow and generates reports covering resource utilization, timing, and power consumption in the `HLS_output/Synthesis/vivado_flow/` directory.

Since this is a simple example designed to validate the complete workflow, a detailed analysis of the generated reports is out of the scope of this section. The full evaluation will be carried out in Section 4, where the workflow is applied to the GPU4S_Bench benchmark suite.

4 GPU4S_Bench

The GPU4S_Bench suite is an open-source collection of benchmarks covering computational kernels representative of space applications, including signal processing, neural network operations, and linear algebra. In this section, the HLS workflow is applied to each benchmark by synthesizing the C/C++ implementation with Bambu and implementing the result in Vivado to obtain resource utilization, timing, and power consumption metrics. A description of each reported metric is provided in Appendix A.

Note For benchmarks requiring source modifications, a copy of the original file was created under the name `cpu_functions_bambu.cpp`.

4.1 LRN_bench

This benchmark implements the Local Response Normalization (LRN) function, commonly used in neural network inference. The HLS synthesis target is the `lrn` function.

Compilation Issues and Fixes

Fix 1: Undefined type `bench_t`. The type `bench_t` is defined in `benchmark_library.h`, located in the parent directory `LRN_bench/`, via preprocessor macros:

```
#ifdef INT
    typedef int bench_t;
#elif FLOAT
    typedef float bench_t;
#elif DOUBLE
    typedef double bench_t;
#else
    /* bench_t undefined */
#endif
```

Since no macro was passed to Bambu, none of the branches were activated and `bench_t` remained undefined, causing the compilation to fail. The fix requires no source code changes, it is sufficient to pass the `-DFLOAT` flag on the command line.

The `-DINT` option is not suitable in this case because the algorithms inherently rely on real-valued computations. Defining `bench_t` as an integer type would force inputs, outputs, and intermediate buffers to be stored as integers, causing truncation and a significant loss of precision.

Fix 2: `pow()` not supported by Bambu. The original `lrn` function computes the LRN formula using `pow()` from `<cmath>`:

Listing 3: Original lrn (cpu_functions.cpp)

```
void lrn(const bench_t* A, bench_t* B, const unsigned int size)
{
    // ...
    B[i*size+j] = A[i*size+j] / pow((K + ALPHA * pow(A[i*size+j], 2)),
        BETA);
    // ...
}
```

Bambu has no functional unit for `pow()` in its HLS resource library, neither in 64-bit nor 32-bit precision. The only supported operations are addition, subtraction, multiplication, and division.

The fix rewrites the computation by decomposing the exponentiation into a combination of roots, each approximated iteratively using the Newton-Raphson method.

Listing 4: Modified lrn (cpu_functions_bambu.cpp)

```
void lrn(const bench_t* A, bench_t* B, const unsigned int size)
{
    // ...
    float a = A[i*size+j];
    float base = 2.0f + 10e-4f * a * a;
    // Newton-Raphson: sqrt(base) = base^0.5
    float y = base * 0.5f;
    y = 0.5f * (y + base / y);
    y = 0.5f * (y + base / y);
    y = 0.5f * (y + base / y);
    // Newton-Raphson: sqrt(y) = base^0.25
    float z = y * 0.5f;
    z = 0.5f * (z + y / z);
    z = 0.5f * (z + y / z);
    z = 0.5f * (z + y / z);
    // base^0.75 = base^0.5 * base^0.25
    float base075 = y * z;
    B[i*size+j] = a / base075;
    // ...
}
```

The final working command is:

```
$ bambu cpu_functions.cpp --top-fname=lrn -DFLOAT
```

Where `-DFLOAT` activates the `float` branch of the conditional typedef, resolving `bench_t` to `float` (32-bit).

Synthesis Results

Bambu successfully completed HLS synthesis of the `lrn` function, generating a Verilog netlist that was subsequently synthesized, placed and routed in Vivado. A summary of the resource utilization and timing results is shown in

Resource	Used	Available	Utilization (%)
LUT	1662	53200	3.12
LUTRAM	0	17400	0.00
FF	1750	106400	1.64
BRAM	0	140	0.00
DSP	5	220	2.27
Frequency (MHz)	103.5, period = 10 ns		
WNS	0.340 ns		

Table 1: Vivado results for `lrn` (Zynq-7020).

4.2 cifar_10

This benchmark implements a full CIFAR-10 inference pipeline, consisting of two convolutional blocks followed by two fully connected layers and a softmax output. The HLS synthesis target is the `cifar10`, which internally calls several functions such as `lrn` and `softmax`.

Compilation Issues and Fixes

Fix 1: Undefined type `bench_t`. As in the previous benchmark, the type `bench_t` is defined conditionally in `benchmark_library.h`. This error is fixed by passing the `-DFLOAT` flag on the command line. This is already explained in Section 4.1. The `-DINT` option is not suitable in this case because the algorithms inherently rely on real-valued computations.

Fix 2: `pow()` in `lrn`. Since `cifar10` calls `lrn` internally, the same `pow()` issue described in the previous benchmark applies here. The identical Newton-Raphson replacement was applied. This fix is already explained in detail in Section 4.1.

Fix 3: `expf()` in `softmax`. The `softmax` function computes the exponential of each input element with the `exp()` function, which is not supported by Bambu:

Listing 5: Original softmax (cpu_functions.cpp)

```
void softmax(const bench_t *A, bench_t *B, const int size)
{
    // ...
    value = expf(A[i]);
    // ...
}
```

The fix replaces it with a fourth-order Taylor series approximation, which only requires the four arithmetic operations supported by Bambu:

Listing 6: Modified softmax (cpu_functions_bambu.cpp)

```
void softmax(const bench_t *A, bench_t *B, const int size)
{
    // ...
    float x = A[i];
```

```
    // e^x approximated by Taylor series: 1 + x + x^2/2 + x^3/6 + x^4/24
    value = 1.0f + x + x*x*0.5f + x*x*x*0.16667f + x*x*x*x*0.04167f;
    // ...
}
```

The final working command is:

```
$ bambu cpu_functions_bambu.cpp --top-fname=cifar10 -DFLOAT
```

Synthesis Results

Bambu successfully completed HLS synthesis of the `cifar10` function, generating a Verilog netlist that was subsequently synthesized, placed and routed in Vivado. A summary of the resource utilization and timing results is shown in Table 2.

Resource	Used	Available	Utilization (%)
LUT	9698	53200	18.23
LUTRAM	0	17400	0.00
FF	7975	106400	7.50
BRAM	0	140	0.00
DSP	126	220	57.27
Frequency (MHz)	100.0, period = 10 ns		
WNS	-0.215 ns		

Table 2: Vivado results for `cifar10` from `cifar_10` (Zynq-7020).

4.3 cifar_10_multiple

The `cifar_10_multiple` benchmark extends the original `cifar_10` implementation by processing multiple input images within a single function call. It introduces an outer loop over the number of images, while keeping the internal CNN computation unchanged. The HLS synthesis target is the `cifar10` function.

Compilation Issues and Fixes

Fix 1: Undefined type `bench_t`. As in the previous benchmarks, the type `bench_t` is defined conditionally in `benchmark_library.h`. This error is fixed by passing the `-DFLOAT` flag on the command line. This is already explained in Section 4.1. The `-DINT` option is not suitable in this case because the algorithms inherently rely on real-valued computations.

Fix 2: `pow()` in `lrn`. Since `cifar10` calls `lrn` internally, the same `pow()` issue described in the previous benchmark applies here. The identical Newton-Raphson replacement was applied. This is already explained in Section 4.1.

Fix 3: `expf()` in `softmax`. The `softmax` function is also called where `expf()` is used as in the previous benchmark. The fourth-order Taylor series approximation was also applied here for its replacement. This is already explained in Section 4.2.

The final working command line is:

```
$ bambu cpu_functions_bambu.cpp --top-fname=cifar10 -DFLOAT
```

Synthesis Results

Bambu successfully completed HLS synthesis of the `cifar10` function, generating a Verilog netlist that was subsequently synthesized, placed and routed in Vivado. A summary of the resource utilization and timing results is shown in Table 3.

Resource	Used	Available	Utilization (%)
LUT	9860	53200	18.53
LUTRAM	0	17400	0.00
FF	8101	106400	7.61
BRAM	0	140	0.00
DSP	132	220	60.00
Frequency (MHz)	100.0, period = 10 ns		
WNS	-0.576 ns		

Table 3: Vivado results for `cifar10` from `cifar_10_multiple` (Zynq-7020).

4.4 convolution_2D_bench

The `convolution_2D_bench` benchmark computes a 2D convolution between an input matrix and a kernel, applying zero-padding at the borders. The HLS synthesis target is the `matrix_convolution` function.

Compilation Issues and Fixes

Fix 1: Undefined type `bench_t`. As in the previous benchmarks, the type `bench_t` is defined conditionally in `benchmark_library.h`. This error is fixed by passing the `-DFLOAT` (or `-DINT` or `-DDOUBLE` flag on the command line. This is already explained in Section 4.1.

The final working commands are:

```
$ bambu cpu_functions.cpp --top-fname=matrix_convolution -DINT
$ bambu cpu_functions.cpp --top-fname=matrix_convolution -DFLOAT
$ bambu cpu_functions.cpp --top-fname=matrix_convolution -DDOUBLE
```

Synthesis Results

Bambu successfully completed HLS synthesis of the `matrix_convolution` function, generating a Verilog netlist that was subsequently synthesized, placed and routed in Vivado. A summary of the resource utilization and timing results is shown in Table 4.

Resource	Available	-DINT		-DFLOAT		-DDOUBLE	
		Used	Util. (%)	Used	Util. (%)	Used	Util. (%)
LUT	53200	972	1.83	1669	3.14	2536	4.77
LUTRAM	17400	0	0.00	0	0.00	0	0.00
FF	106400	627	0.59	1277	1.20	1812	1.70
BRAM	140	0	0.00	0	0.00	0	0.00
DSP	220	12	5.45	11	5.00	19	8.64
Frequency (MHz)	–	100.0 (10 ns)		100.0 (10 ns)		100.0 (10 ns)	
WNS (ns)	–	3.274		0.091		-0.200	

Table 4: Vivado results for `matrix_convolution` (Zynq-7020).

4.5 correlation_2D

This benchmark computes the correlation coefficient between two matrices. The HLS target is the `correlation_2D` function, which calculates the mean values and performs the final normalized accumulation.

Compilation Issues and Fixes

Fix 1: Undefined type `bench_t`. As in the previous benchmarks, the type `bench_t` is defined conditionally in `benchmark_library.h`. This error is fixed by passing the `-DFLOAT` flag on the command line. This is already explained in Section 4.1. The `-DINT` option is not suitable in this case because the algorithm relies on real-valued operations such as mean computation, normalization, square root evaluation, and a final division that typically produces fractional results.

Fix 2: `sqrt` in `correlation_2D`. The original implementation computes the normalization term using `sqrt()`, which is not supported by Bambu, as it has no corresponding functional unit in its HLS resource library.

Listing 7: Original correlation_2D (cpu_functions_bambu.cpp)

```
void correlation_2D(const bench_t* A, const bench_t* B, result_bench_t*
    R ,const int size){
     // ...
  *R = (result_bench_t)(acumulate_value_a_b / (result_bench_t)(sqrt(
      acumulate_value_a_a * acumulate_value_b_b)));
}
```

To solve this issue, `sqrt()` was replaced with a custom implementation based on the Newton-Raphson method. This approximation only uses basic arithmetic operations:

Listing 8: Modified correlation_2D (cpu_functions_bambu.cpp)

```
// sqrt approximation
static result_bench_t bambu_sqrt(result_bench_t x){
    if (x <= 0) return 0;
    result_bench_t guess = x;
    for (int i = 0; i < 20; i++){
        guess = 0.5f * (guess + x / guess);
    }
    return guess;
}

void correlation_2D(const bench_t* A, const bench_t* B, result_bench_t*
    R ,const int size){
    // ...
    // Modification --> call the sqrt approximation
  result_bench_t denom = bambu_sqrt(acumulate_value_a_a *
      acumulate_value_b_b);
  if (denom == 0)
        *R = 0;
  else
        *R = (result_bench_t)(acumulate_value_a_b / denom);
}
```

The final working command is:

```
$ bambu cpu_functions_bambu.cpp --top-fname=correlation_2D -DFLOAT
```

Synthesis Results

Bambu successfully completed HLS synthesis of the `correlation_2D` function, generating a Verilog netlist that was subsequently synthesized, placed and routed in Vivado. A summary of the resource utilization and timing results is shown in Table 5.

Resource	Used	Available	Utilization (%)
LUT	3791	53200	7.13
LUTRAM	0	17400	0.00
FF	2902	106400	2.73
BRAM	0	140	0.00
DSP	14	220	6.36
Frequency (MHz)	100.0, period = 10 ns		
WNS	-0.219 ns		

Table 5: Vivado results for `correlation_2D` (float 32-bit, Zynq-7020).

4.6 fast_fourier_transform_2D_bench

This benchmark implements a 2-D discrete Fourier transform of a complex-valued input matrix of size $nx \times ny$. The HLS synthesis target is the `FFT2D` function, which originally delegates the computation to the `fftw_plan_dft_2d` routine from the FFTW3 library.

Compilation Issues and Fixes

Fix 1: Undefined type `bench_t`. As in the previous benchmarks, the type `bench_t` is defined conditionally in `benchmark_library.h`. This error is fixed by passing the `-DFLOAT` flag on the command line. This is already explained in Section 4.1. The `-DINT` option is not suitable for this case because the algorithm relies on trigonometric coefficients, complex-valued rotations, and fractional intermediate values.

Fix 2: Library `fftw3` not supported by Bambu. The original implementation relies entirely on the FFTW3 library for the FFT computation, which is not supported by Bambu since it cannot link against external pre-compiled libraries.

Listing 9: Original FFT2D (cpu_functions.cpp)

```
#include <fftw3.h>

bool FFT2D(COMPLEX **c, int nx, int ny, COMPLEX **out)
{
    fftw_plan plan;
    fftw_complex *in = (fftw_complex*) fftw_malloc(sizeof(fftw_complex)*
       nx*ny);
    fftw_complex *out_data = (fftw_complex*) fftw_malloc(sizeof(
       fftw_complex)*nx*ny);
    /* ... fill in ... */
    plan = fftw_plan_dft_2d(nx, nx, in, out_data, FFTW_FORWARD,
       FFTW_ESTIMATE);
    fftw_execute(plan);
    /* ... copy out ... */
    return true;
}
```

Bambu requires that all code to be synthesised is available as plain C/C++ source. The entire `FFT2D` function was therefore rewritten from scratch.

The reimplementation is based on the Cooley–Tukey radix-2 algorithm, which decomposes an N-point DFT into two $N/2$-point DFTs and requires only the four basic arithmetic operations together with trigonometric twiddle factors. The 2-D transform is obtained by applying the 1-D transform first to every row and then to every column of the matrix, using a statically allocated column buffer to avoid dynamic memory allocation:

Listing 10: Modified FFT2D (cpu_functions_bambu.cpp)

```
#define MAX_N 64

static void fft1D(COMPLEX* row, int n) {
    // Bit-reversal permutation
    int j = 0;
    for (int i = 1; i < n; i++) {
        int bit = n >> 1;
        for (; j & bit; bit >>= 1) j ^= bit;
        j ^= bit;
        if (i < j) {
            bench_t tmp_x = row[i].x; row[i].x = row[j].x; row[j].x =
               tmp_x;
            bench_t tmp_y = row[i].y; row[i].y = row[j].y; row[j].y =
               tmp_y;
        }
    }
    // Cooley-Tukey butterfly
    for (int len = 2; len <= n; len <<= 1) {
        bench_t ang  = -6.28318530717958647692 / (bench_t)len;
        bench_t wcos = bambu_cos(ang);
        bench_t wsin = bambu_sin(ang);
        for (int i = 0; i < n; i += len) {
            bench_t cur_cos = 1.0, cur_sin = 0.0;
            for (int k = 0; k < len / 2; k++) {
                bench_t ux = row[i+k].x, uy = row[i+k].y;
                bench_t vx = row[i+k+len/2].x*cur_cos - row[i+k+len/2].y
                   *cur_sin;
                bench_t vy = row[i+k+len/2].x*cur_sin + row[i+k+len/2].y
                   *cur_cos;
                row[i+k].x       = ux + vx;  row[i+k].y       = uy + vy;
                row[i+k+len/2].x = ux - vx;  row[i+k+len/2].y = uy - vy;
                bench_t nc = cur_cos*wcos - cur_sin*wsin;
                bench_t ns = cur_cos*wsin + cur_sin*wcos;
                cur_cos = nc; cur_sin = ns;
            }
        }
    }
}

bool FFT2D(COMPLEX **c, int nx, int ny, COMPLEX **out) {
    for (int i = 0; i < nx; i++)
        for (int j = 0; j < ny; j++)
            out[i][j] = c[i][j];
    for (int i = 0; i < nx; i++)
        fft1D(out[i], ny);
    COMPLEX col[MAX_N];
    for (int j = 0; j < ny; j++) {
        for (int i = 0; i < nx; i++) col[i] = out[i][j];
```

```
45          fft1D(col, nx);
46          for (int i = 0; i < nx; i++) out[i][j] = col[i];
47      }
48      return true;
49  }
```

The macro `MAX_N` must be set to at least the largest matrix dimension used in the benchmark; it defaults to 64. This implementation requires nx and ny to be powers of two, which is the standard assumption for radix-2 FFT benchmarks.

Fix 3: `cos()` and `sin()` not supported by Bambu. The Cooley–Tukey algorithm requires the evaluation of trigonometric twiddle factors $\cos(\theta)$ and $\sin(\theta)$. Bambu cannot synthesise calls to `cos()` or `sin()` from `<math.h>`.

Both functions were therefore replaced with fixed-iteration Taylor series approximations. The angle is first reduced to $[-\pi, \pi]$ to ensure convergence, and then 12 terms of the series are accumulated using only the four arithmetic operations:

Listing 11: Trigonometric approximations for *sin*() and *cos*() (cpu_functions_bambu.cpp)

```
1  static bench_t bambu_mod_pi(bench_t x) {
2      bench_t two_pi = 6.28318530717958647692;
3      bench_t pi     = 3.14159265358979323846;
4      for (int i = 0; i < 20; i++) {
5          if (x >  pi) x = x - two_pi;
6          if (x < -pi) x = x + two_pi;
7      }
8      return x;
9  }
10
11 // sin(x) = x - x^3/3! + x^5/5! - ... (12 terms)
12 static bench_t bambu_sin(bench_t x) {
13     x = bambu_mod_pi(x);
14     bench_t term = x, result = x;
15     for (int i = 1; i < 12; i++) {
16         bench_t denom = (bench_t)((2*i) * (2*i + 1));
17         term = -term * (x * x) / denom;
18         result = result + term;
19     }
20     return result;
21 }
22
23 // cos(x) = 1 - x^2/2! + x^4/4! - ... (12 terms)
24 static bench_t bambu_cos(bench_t x) {
25     x = bambu_mod_pi(x);
26     bench_t term = 1.0, result = 1.0;
27     for (int i = 1; i < 12; i++) {
28         bench_t denom = (bench_t)((2*i - 1) * (2*i));
29         term = -term * (x * x) / denom;
30         result = result + term;
31     }
32     return result;
33 }
```

Note that Listing 10 is already implemented with the `bambu_cos` and `bambu_sin` functions.

The final working command is:

```
$ bambu cpu_functions_bambu.cpp --top-fname=FFT2D -DFLOAT
```

Synthesis Results

Bambu successfully completed HLS synthesis of the `FFT2D` function, generating a Verilog netlist that was subsequently synthesized, placed and routed in Vivado. A summary of the resource utilization and timing results is shown in Table 6.

Resource	Used	Available	Utilization (%)
LUT	6894	53200	12.96
LUTRAM	0	17400	0.00
FF	5044	106400	4.74
BRAM	1	140	0.71
DSP	8	220	3.64
Frequency (MHz)	100.0, period = 10 ns		
WNS	-1.064 ns		

Table 6: Vivado results for `FFT2D` (Zynq-7020).

4.7 fast_fourier_transform_bench

This benchmark implements a 1-D discrete Fourier transform of a real-valued input array using the Danielson–Lanczos variant of the Cooley–Tukey radix-2 algorithm, directly in C++ without relying on any external FFT library.

Compilation Issues and Fixes

Fix 1: Undefined type `bench_t`. As in the previous benchmarks, the type `bench_t` is defined conditionally in `benchmark_library.h`. This error is fixed by passing the `-DFLOAT` flag on the command line. This is already explained in Section 4.1. The `-DINT` option is not suitable in this case because the algorithm relies on trigonometric coefficients, fractional angle values, and complex-valued rotations during the butterfly stages.

Fix 2: `sin()` and `M_PI` not supported by Bambu. The `fft_function` computes the twiddle factors using two calls to `sin()` and the constant `M_PI`:

Listing 12: Original fft_function (cpu_functions.cpp)

```
void fft_function(bench_t* data, int64_t nn){
    // ...
    theta = -(2*M_PI/mmax);
    wtemp = sin(0.5*theta);
    wpr   = -2.0*wtemp*wtemp;
    wpi   = sin(theta);
    // ...
}
```

The fix reuses the `bambu_mod_pi` and `bambu_sin` Taylor series approximations developed in Section 4.6, and replaces `M_PI` with its literal value to avoid any dependency on `<math.h>`.

Listing 13: Modified fft_function (cpu_functions_bambu.cpp)

```
void fft_function(bench_t* data, int64_t nn){
    // ...
```

```
    theta = -(2*3.14159265358979323846/mmax);
    wtemp = bambu_sin(0.5*theta);
    wpr   = -2.0*wtemp*wtemp;
    wpi   = bambu_sin(theta);
    // ...
}
```

The final working command is:

```
$ bambu cpu_functions_bambu.cpp --top-fname=fft_function -DFLOAT
```

Synthesis Results

Bambu successfully completed HLS synthesis of the `fft_function` function, generating a Verilog netlist that was subsequently synthesized, placed and routed in Vivado. A summary of the resource utilization and timing results is shown in Table 7.

Resource	Used	Available	Utilization (%)
LUT	7719	53200	14.51
LUTRAM	0	17400	0.00
FF	5725	106400	5.38
BRAM	0	140	0.00
DSP	18	220	8.18
Frequency (MHz)	100.0, period = 10 ns		
WNS	-0.689 ns		

Table 7: Vivado results for `fft_function` from `fast_fourier_transform_bench` (Zynq-7020).

4.8 fast_fourier_transform_window_bench

This benchmark computes a windowed FFT over the input signal. The HLS target is the `fft_function`, which copies each input window to the output buffer and then applies the FFT.

Compilation Issues and Fixes

Fix 1: Undefined type `bench_t`. As in the previous benchmarks, the type `bench_t` is defined conditionally in `benchmark_library.h`. This error is fixed by passing the `-DFLOAT` flag on the command line. This is already explained in Section 4.1. The `-DINT` option is not suitable in this case because the algorithm relies on trigonometric coefficients, fractional angle values, and complex-valued rotations during the butterfly stages.

Fix 2: `sin()` and `M_PI` not supported by Bambu. As in the previous benchmarks, the `fft_function` computes the twiddle factors using two calls to `sin()` and the constant `M_PI`. This is fixed the same way as Section 4.6 and Section 4.7.

The final working command is:

```
$ bambu cpu_functions_bambu.cpp --top-fname=fft_function -DFLOAT
```

Synthesis Results

Bambu successfully completed HLS synthesis of the `fft_function` function, generating a Verilog netlist that was subsequently synthesized, placed and routed in Vivado. A summary of the resource utilization and timing results is shown in Table 8.

Resource	Used	Available	Utilization (%)
LUT	8230	53200	15.47
LUTRAM	0	17400	0.00
FF	6032	106400	5.67
BRAM	0	140	0.00
DSP	24	220	10.91
Frequency (MHz)	100.0, period = 10 ns		
WNS	-0.325 ns		

Table 8: Vivado results for `fft_function` from `fast_fourier_transform_window_bench` (Zynq-7020).

4.9 finite_impulse_response_filter

This benchmark computes a Finite Impulse Response (FIR) filter over an input signal. The HLS target is the `vector_convolution` function, which slides the filter kernel over the input signal and accumulates the weighted sum of samples at each output position.

Compilation Issues and Fixes

For this benchmark, no compilation issues were reported. Both the `-DINT` and `-DFLOAT` options can be used, since the FIR computation only involves additions and multiplications. By default, the data type is defined as `double`.

The working commands are:

```
$ bambu cpu_functions.cpp --top-fname=vector_convolution -DINT
$ bambu cpu_functions.cpp --top-fname=vector_convolution -DFLOAT
$ bambu cpu_functions.cpp --top-fname=vector_convolution
```

Synthesis Results

Bambu successfully completed HLS synthesis of the `vector_convolution` function, generating a Verilog netlist that was subsequently synthesized, placed and routed in Vivado. A summary of the resource utilization and timing results is shown in Table 9.

Resource	Available	-DINT		-DFLOAT		-DDOUBLE	
		Used	Util. (%)	Used	Util. (%)	Used	Util. (%)
LUT	53200	530	1.00	933	1.75	1680	3.16
LUTRAM	17400	0	0.00	0	0.00	0	0.00
FF	106400	443	0.42	727	0.68	1293	1.22
BRAM	140	0	0.00	0	0.00	0	0.00
DSP	220	3	1.36	2	0.91	10	4.55
Frequency (MHz)	–	100.0 (10 ns)		100.0 (10 ns)		100.0 (10 ns)	
WNS (ns)	–	4.039		0.184		-0.464	

Table 9: Vivado results for `vector_convolution` (Zynq-7020).

4.10 matrix_multiplication_bench

This benchmark computes the multiplication of two square matrices of size $N \times N$. The HLS target is the `matrix_multiplication` function, which implements the standard triple-nested loop algorithm accumulating the dot product of rows and columns into the output matrix `C`.

Compilation Issues and Fixes

Fix 1: Undefined type `bench_t`. As in previous benchmarks, the type `bench_t` is defined conditionally in `benchmark_library.h`. This error is fixed by passing the `-DFLOAT` (or `-DINT` or `-DDOUBLE`) flag on the command line. This is already explained in Section 4.1.

The working commands are:

```
$ bambu cpu_functions.cpp --top-fname=matrix_multiplication -DINT
$ bambu cpu_functions.cpp --top-fname=matrix_multiplication -DFLOAT
$ bambu cpu_functions.cpp --top-fname=matrix_multiplication -DDOUBLE
```

Synthesis Results

Bambu successfully completed HLS synthesis of the `matrix_multiplication` function, generating a Verilog netlist that was subsequently synthesized, placed and routed in Vivado. A summary of the resource utilization and timing results is shown in Table 10.

Resource	**Available**	**-DINT**		**-DFLOAT**		**-DDOUBLE**	
		Used	Util. (%)	Used	Util. (%)	Used	Util. (%)
LUT	53200	536	1.01	913	1.72	1687	3.17
LUTRAM	17400	0	0.00	0	0.00	0	0.00
FF	106400	258	0.24	574	0.54	1071	1.01
BRAM	140	0	0.00	0	0.00	0	0.00
DSP	220	9	4.09	8	3.64	16	7.27
Frequency (MHz)	–	100.0 (10 ns)		100.0 (10 ns)		100.0 (10 ns)	
WNS (ns)	–	3.508		0.214		0.017	

Table 10: Vivado results for `matrix_multiplication` from `matrix_multiplication_bench` (Zynq-7020).

4.11 matrix_multiplication_bench_fp16

This benchmark computes the multiplication of two square matrices, analogously to the previous `matrix_multiplication_bench`. The key difference is that this variant targets half-precision floating-point (FP16) arithmetic, intended for GPU execution using CUDA `half` types. The HLS target is again the `matrix_multiplication` function.

Compilation Issues and Fixes

For this benchmark, no compilation issues were reported.

The final working commands are:

```
$ bambu lib_cpu.cpp --top-fname=matrix_multiplication -DINT
$ bambu lib_cpu.cpp --top-fname=matrix_multiplication -DFLOAT
$ bambu lib_cpu.cpp --top-fname=matrix_multiplication -DDOUBLE
```

Although `-DINT` is functionally valid but does not accurately represent the intended floating-point computation, as it removes fractional precision. In contrast, `-DFLOAT` provides the closest practical approximation to FP16 behavior within the HLS framework, while `-DDOUBLE` represents a higher-precision baseline.

Synthesis Results

Bambu successfully completed HLS synthesis of the `matrix_multiplication` function, generating a Verilog netlist that was subsequently synthesized, placed and routed in Vivado. A summary of the resource utilization and timing results is shown in Table 11.

Resource	**Available**	**-DINT**		**-DFLOAT**		**-DDOUBLE**	
		Used	Util. (%)	Used	Util. (%)	Used	Util. (%)
LUT	53200	536	1.01	913	1.72	1687	3.17
LUTRAM	17400	0	0.00	0	0.00	0	0.00
FF	106400	258	0.24	574	0.54	1071	1.01
BRAM	140	0	0.00	0	0.00	0	0.00
DSP	220	9	4.09	8	3.64	16	7.27
Frequency (MHz)	–	100.0 (10 ns)		100.0 (10 ns)		100.0 (10 ns)	
WNS (ns)	–	3.508		0.214		0.017	

Table 11: Vivado results for `matrix_multiplication` from `matrix_multiplication_bench_fp16` (Zynq-7020).

4.12 matrix_multiplication_tensor_bench

This benchmark computes the multiplication of two square matrices, analogously to the previous `matrix_multiplication_bench` and `matrix_multiplication_bench_fp16`. The key difference is that this variant targets GPU tensor cores, which provide specialized hardware units for mixed-precision matrix operations. The HLS target is again the `matrix_multiplication` function.

Compilation Issues and Fixes

Fix 1: Undefined type `bench_t`. As in the previous benchmarks, the type `bench_t` is defined conditionally in `benchmark_library.h`. This error is fixed by passing the `-DFLOAT` (or `-DINT` or `-DDOUBLE`) flag on the command line. This is already explained in Section 4.1.

The final working commands are:

```
$ bambu lib_cpu.cpp --top-fname=matrix_multiplication -DINT
$ bambu lib_cpu.cpp --top-fname=matrix_multiplication -DFLOAT
$ bambu lib_cpu.cpp --top-fname=matrix_multiplication -DDOUBLE
```

Synthesis Results

Bambu successfully completed HLS synthesis of the `matrix_multiplication` function, generating a Verilog netlist that was subsequently synthesized, placed and routed in Vivado. A summary of the resource utilization and timing results is shown in Table 12.

Resource	Available	-DINT		-DFLOAT		-DDOUBLE	
		Used	Util. (%)	Used	Util. (%)	Used	Util. (%)
LUT	53200	536	1.01	913	1.72	1687	3.17
LUTRAM	17400	0	0.00	0	0.00	0	0.00
FF	106400	258	0.24	574	0.54	1071	1.01
BRAM	140	0	0.00	0	0.00	0	0.00
DSP	220	9	4.09	8	3.64	16	7.27
Frequency (MHz)	–	100.0 (10 ns)		100.0 (10 ns)		100.0 (10 ns)	
WNS (ns)	–	3.508		0.214		0.017	

Table 12: Vivado results for `matrix_multiplication` from `matrix_multiplication_tensor_bench` (Zynq-7020).

This benchmark produces identical RTL output to `matrix_multiplication_bench`. This is expected: the tensor core distinction exists exclusively at the GPU execution level, where NVIDIA tensor cores provide specialized mixed-precision hardware units. The CPU reference implementation in both cases resolves to the same `float`-precision `matrix_multiplication` function, and Bambu therefore produces an identical netlist.

4.13 max_pooling_bench

This benchmark computes a max pooling operation over a 2D input matrix, a fundamental building block in convolutional neural networks used to reduce spatial dimensions while retaining the most prominent features. The HLS target is the `max_pooling` function, which slides a window of size `stride` × `stride` over the input matrix and writes the maximum value of each window to the output matrix.

Compilation Issues and Fixes

Fix 1: Undefined type `bench_t`. As in the previous benchmarks, the type `bench_t` is defined conditionally in `benchmark_library.h`. This error is fixed by passing the `-DFLOAT` (or `-DINT` or `DDOUBLE`) flag on the command line. This is already explained in Section 4.1.

The final working commands are:

```
$  bambu cpu_functions.cpp --top-fname=max_pooling -DINT
$  bambu cpu_functions.cpp --top-fname=max_pooling -DFLOAT
$  bambu cpu_functions.cpp --top-fname=max_pooling -DDOUBLE
```

Synthesis Results

Bambu successfully completed HLS synthesis of the `max_pooling` function, generating a Verilog netlist that was subsequently synthesized, placed and routed in Vivado. A summary of the resource utilization and timing results is shown in Table 13.

Resource	Available	-DINT		-DFLOAT		-DDOUBLE	
		Used	Util. (%)	Used	Util. (%)	Used	Util. (%)
LUT	53200	1364	2.56	1329	2.50	1465	2.75
LUTRAM	17400	0	0.00	0	0.00	0	0.00
FF	106400	916	0.86	883	0.83	1024	0.96
BRAM	140	0	0.00	0	0.00	0	0.00
DSP	220	33	15.00	33	15.00	33	15.00
Frequency (MHz)	–	100.0 (10 ns)		100.0 (10 ns)		100.0 (10 ns)	
WNS (ns)	–	-0.240		-0.225		-0.186	

Table 13: Vivado results for `max_pooling` (Zynq-7020).

4.14 memory_bandwidth_bench

This benchmark measures memory bandwidth by performing a device-to-device memory copy via `cudaMemcpy`, with no synthesisable computational kernel in the original source. Following feedback from the project supervisor, a plain-C `copy_array` function was implemented from scratch as the HLS target.

Compilation Issues and Fixes

Fix 1: No synthesisable HLS target in the original source. As previously mentioned, the original benchmark contains no CPU-side computational kernel that can be passed to Bambu. The `execute_kernel` function relies exclusively on `cudaMemcpy`, which Bambu cannot synthesise. The fix is to implement the copy operation directly in C. A new `cpu_functions/` subdirectory was created containing two files.

`cpu_functions/cpu_functions.h` declares the `bench_t` type and the function prototype:

Listing 14: Created header cpu_functions.h

```
#ifndef CPU_FUNCTIONS_H
#define CPU_FUNCTIONS_H

#ifdef INT
  typedef int bench_t;
#elif FLOAT
  typedef float bench_t;
#elif DOUBLE
  typedef double bench_t;
#endif

void copy_array(bench_t *A, bench_t *B, int size);

#endif
```

`cpu_functions/cpu_functions.cpp` provides the implementation:

Listing 15: copy_array function (cpu_functions.cpp)

```
#include "cpu_functions.h"

void copy_array(bench_t *A, bench_t *B, int size) {
    for (int i = 0; i < size; i++) {
        B[i] = A[i];
    }
}
```

Fix 2: Undefined type `bench_t`. As in the previous benchmarks, the type `bench_t` is defined conditionally via preprocessor macros. This error is fixed by passing the `-DFLOAT` (or `-DINT` or `-DDOUBLE`) flag on the command line. This is already explained in Section 4.1.

The final working commands are:

```
$ bambu cpu_functions.cpp --top-fname=copy_array -DINT
$ bambu cpu_functions.cpp --top-fname=copy_array -DFLOAT
$ bambu cpu_functions.cpp --top-fname=copy_array -DDOUBLE
```

Synthesis Results

Bambu successfully completed HLS synthesis of the `copy_array` function, generating a Verilog netlist that was subsequently synthesized, placed and routed in Vivado. A summary of the resource utilization and timing results is shown in Table 14.

Resource	**Available**	**-DINT**		**-DFLOAT**		**-DDOUBLE**	
		Used	Util. (%)	Used	Util. (%)	Used	Util. (%)
LUT	53200	162	0.30	162	0.30	176	0.33
LUTRAM	17400	0	0.00	0	0.00	0	0.00
FF	106400	104	0.10	104	0.10	136	0.13
BRAM	140	0	0.00	0	0.00	0	0.00
DSP	220	0	0.00	0	0.00	0	0.00
Frequency (MHz)	–	100.0 (10 ns)		100.0 (10 ns)		100.0 (10 ns)	
WNS (ns)	–	5.311		5.311		5.150	

Table 14: Vivado results for `copy_array` (Zynq-7020).

4.15 relu_bench

This benchmark computes the Rectified Linear Unit (ReLU) activation function over a 2D input matrix. The HLS target is the `relu` function, which sets each element of the output matrix to the input value if positive, and to zero otherwise.

Compilation Issues and Fixes

Fix 1: Undefined type `bench_t`. As in the previous benchmarks, the type `bench_t` is defined conditionally in `benchmark_library.h`. This error is fixed by passing the `-DFLOAT` (or `-DINT` or `DDOUBLE`) flag on the command line. This is already explained in Section 4.1.

The final working commands are:

```
$  bambu cpu_functions.cpp --top-fname=relu -DINT
$  bambu cpu_functions.cpp --top-fname=relu -DFLOAT
$  bambu cpu_functions.cpp --top-fname=relu -DDOUBLE
```

Synthesis Results

Bambu successfully completed HLS synthesis of the `relu` function, generating a Verilog netlist that was subsequently synthesized, placed and routed in Vivado. A summary of the resource utilization and timing results is shown in Table 15.

Resource	Available	-DINT		-DFLOAT		-DDOUBLE	
		Used	Util. (%)	Used	Util. (%)	Used	Util. (%)
LUT	53200	222	0.42	298	0.56	373	0.70
LUTRAM	17400	0	0.00	0	0.00	0	0.00
FF	106400	153	0.14	153	0.14	196	0.18
BRAM	140	0	0.00	0	0.00	0	0.00
DSP	220	3	1.36	3	1.36	3	1.36
Frequency (MHz)	–	100.0 (10 ns)		100.0 (10 ns)		100.0 (10 ns)	
WNS (ns)	–	4.536		4.462		2.337	

Table 15: Vivado results for `relu` (Zynq-7020).

4.16 softmax_bench

This benchmark computes the Softmax activation function over a 2D input matrix, a fundamental operation in neural networks used to normalize the output of a layer into a probability distribution. The HLS target is the `softmax` function, which computes the exponential of each element, accumulates the total sum, and then normalizes each element by dividing by that sum.

Compilation Issues and Fixes

Fix 1: Undefined type `bench_t`. As in the previous benchmarks, the type `bench_t` is defined conditionally in `cpu_functions.h`. This error is fixed by passing the `-DFLOAT` flag on the command line. This is already explained in Section 4.1. The `-DINT` option is not suitable because the softmax computation relies on exponential evaluation and normalization through division, both of which produce fractional values.

Fix 2: `exp()` not supported by Bambu. The `softmax` function computes the exponential of each element using `exp()`. Taylor series approximation was also applied here for its replacement the same way as in Section 4.2.

The final working command is:

```
$ bambu cpu_functions_bambu.cpp --top-fname=softmax -DFLOAT
```

Synthesis Results

Bambu successfully completed HLS synthesis of the `softmax` function, generating a Verilog netlist that was subsequently synthesized, placed and routed in Vivado. A summary of the resource utilization and timing results is shown in Table 16.

Resource	Used	Available	Utilization (%)
LUT	1476	53200	2.77
LUTRAM	0	17400	0.00
FF	1418	106400	1.33
BRAM	0	140	0.00
DSP	8	220	3.64
Frequency (MHz)	100.0, period = 10 ns		
WNS	0.028 ns		

Table 16: Vivado results for `softmax` (Zynq-7020).

4.17 wavelet_transform

This benchmark computes the CCSDS Wavelet Transform over a 1D input signal, a multi-resolution analysis technique. The HLS target is the `ccsds_wavelet_transform` function, which applies a lowpass and a highpass filter to the input signal, storing the subband coefficients in the lower and upper halves of the output array respectively.

Compilation Issues and Fixes

Fix 1: Undefined type `bench_t`. As in the previous benchmarks, the type `bench_t` is defined conditionally in `cpu_functions.h`. This error is fixed by passing the `-DFLOAT` (or `-DINT` or `DDOUBLE`) flag on the command line. This is already explained in Section 4.1.

The final working commands are:

```
$ bambu cpu_functions.cpp --top-fname=ccsds_wavelet_transform -DINT
$ bambu cpu_functions.cpp --top-fname=ccsds_wavelet_transform -DFLOAT
$ bambu cpu_functions.cpp --top-fname=ccsds_wavelet_transform -DDOUBLE
```

Synthesis Results

Bambu successfully completed HLS synthesis of the `ccsds_wavelet_transform` function, generating a Verilog netlist that was subsequently synthesized, placed and routed in Vivado. A summary of the resource utilization and timing results is shown in Table 17.

Resource	**Available**	**-DINT**		**-DFLOAT**		**-DDOUBLE**	
		Used	Util. (%)	Used	Util. (%)	Used	Util. (%)
LUT	53200	5038	9.47	1328	2.50	2024	3.80
LUTRAM	17400	0	0.00	0	0.00	0	0.00
FF	106400	2971	2.79	1162	1.09	1636	1.54
BRAM	140	0	0.00	0	0.00	0	0.00
DSP	220	2	0.91	2	0.91	10	4.55
Frequency (MHz)	–	100.0 (10 ns)		100.0 (10 ns)		100.0 (10 ns)	
WNS (ns)	–	0.187		0.146		-0.204	

Table 17: Vivado results for `ccsds_wavelet_transform` (Zynq-7020).

5 Results Summary & Discussion

Table 18 shows all the results obtained for each benchmark from the previous section for the `-DFLOAT` version.

	LUT	FF	BRAM	DSP	Freq (MHz)	WNS (ns)
Available	53200	106400	140	220	–	–
`LRN_bench`	1662 (3.12%)	1750 (1.64%)	0 (0.00%)	5 (2.27%)	103.5	0.340
`cifar_10`	9698 (18.23%)	7975 (7.50%)	0 (0.00%)	126 (57.27%)	100.0	-0.215
`cifar_10_multiple`	9860 (18.53%)	8101 (7.61%)	0 (0.00%)	132 (60.00%)	100.0	-0.576
`convolution_2D_bench`	1669 (3.14%)	1277 (1.20%)	0 (0.00%)	11 (5.00%)	100.0	0.091
`correlation_2D`	3791 (7.13%)	2902 (2.73%)	0 (0.00%)	14 (6.36%)	100.0	-0.219
`fft_2D_bench`	6894 (12.96%)	5044 (4.74%)	1 (0.71%)	8 (3.64%)	100.0	-1.064
`fft_bench`	7719 (14.51%)	5725 (5.38%)	0 (0.00%)	18 (8.18%)	100.0	-0.689
`fft_window_bench`	8230 (15.47%)	6032 (5.67%)	0 (0.00%)	24 (10.91%)	100.0	-0.325
`fir_filter`	933 (1.75%)	727 (0.68%)	0 (0.00%)	2 (0.91%)	100.0	0.184
`matrix_mult_bench`	913 (1.72%)	574 (0.54%)	0 (0.00%)	8 (3.64%)	100.0	0.214
`matrix_mult_bench_fp16`	1687 (3.17%)	1071 (1.01%)	0 (0.00%)	16 (7.27%)	100.0	0.017
`matrix_mult_tensor_bench`	913 (1.72%)	574 (0.54%)	0 (0.00%)	8 (3.64%)	100.0	0.214
`max_pooling_bench`	1329 (2.50%)	883 (0.83%)	0 (0.00%)	33 (15.00%)	100.0	-0.225
`memory_bandwidth_bench`	162 (0.3%)	144 (0.1%)	0 (0.00%)	0 (0.00%)	100	5.311
`relu_bench`	298 (0.56%)	153 (0.14%)	0 (0.00%)	3 (1.36%)	100.0	4.462
`softmax_bench`	1476 (2.77%)	1418 (1.33%)	0 (0.00%)	8 (3.64%)	100.0	0.028
`wavelet_transform`	1328 (2.50%)	1162 (1.09%)	0 (0.00%)	2 (0.91%)	100.0	0.146

Table 18: Results summary for all GPU4S_Bench benchmarks on the Zynq-7020 FPGA with the `-DFLOAT` version.

All synthesized benchmarks reported zero LUTRAM utilization, indicating that no distributed memory structures were inferred by HLS. Therefore, we omitted its value from the summary.

Benchmarks excluded from datatypes config. Not all GPU4S_Bench kernels were evaluated under the three datatype configurations (`-DINT`, `-DFLOAT`, and `-DDOUBLE`). In several cases, a datatype variant was intentionally excluded. Table 21 in Appendix B summarizes the excluded configurations and their justification.

5.1 Description of the results

Out of the 17 benchmarks from the GPU4S_Bench suite, all 17 were successfully synthesized using Bambu and implemented in Vivado.

Effect of the datatype configuration

For the benchmarks that support multiple datatype configurations, a clear and consistent trend is observed across all of them: resource utilization and critical path length both increase monotonically from `-DINT` to `-DFLOAT` to `-DDOUBLE`.

In terms of **LUT and FF usage**, integer designs are the most compact. Moving to `-DFLOAT` introduces floating-point datapaths, which require significantly more logic to implement the IEEE 754 arithmetic units inferred by Bambu. Upgrading to `-DDOUBLE` further increases resource consumption, as 64-bit floating-point units are considerably larger than their 32-bit counterparts.

DSP utilization follows a similar trend in most benchmarks, with `-DDOUBLE` consistently requiring more DSP slices than `-DFLOAT`. A notable exception is `max_pooling`, where DSP usage remains constant at 33 across all three variants (15.00%), suggesting that Bambu maps the comparison-heavy pooling logic onto DSP blocks regardless of precision.

Timing degrades with increasing precision. `-DINT` designs consistently achieve the best WNS, often with large positive slack. The `-DFLOAT` variants reduce the slack significantly due to the longer combinational paths through floating-point units, and `-DDOUBLE` pushes the critical path further, causing timing violations in benchmarks that previously met the constraint under `-DFLOAT`.

Comparison across benchmarks (`-DFLOAT`)

Of the 17 benchmarks, **10 meet the timing constraint** (WNS ≥ 0) and **7 present a timing violation** (WNS < 0). The default timing constraint applied by Bambu's generated TCL script targets a clock period of 10 ns (100 MHz), with the exception of `LRN_bench`, for which Bambu inferred a tighter constraint of 9.662 ns (103.5 MHz).

Benchmarks meeting timing. The benchmarks that satisfy the timing constraint are those implementing relatively simple computational kernels:

- `LRN_bench`
- `convolution_2D_bench`
- `fir_filter`
- `matrix_mult_bench`
- `matrix_mult_bench_fp16`
- `matrix_mult_tensor_bench`
- `memory_bandwidth_bench`
- `relu_bench`
- `softmax_bench`
- `wavelet_transform`

These designs share a common characteristic: their logic is dominated by straightforward arithmetic pipelines (multiply-accumulate patterns) for which Bambu can generate efficient RTL without requiring unsupported library functions.

Resource utilization across these benchmarks is generally low. LUT usage ranges from 0.30% to 3.17%, and no benchmark in this group exceeds 8% DSP utilization.

Also note that `matrix_mult_bench`, `matrix_mult_bench_fp16`, and `matrix_mult_tensor_bench` produce identical synthesis results. This is expected, since all three resolve to the same matrix multiplication C function in the CPU reference implementation. The GPU-level distinctions (FP16, tensor cores) have no equivalent in the HLS flow, and Bambu therefore produces an identical netlist in all cases.

Benchmarks with timing violations. The benchmarks that fail to meet the 100 MHz constraint are:

- `cifar_10`
- `cifar_10_multiple`
- `correlation_2D`
- `fft_2D_bench`
- `fft_bench`
- `fft_window_bench`
- `max_pooling_bench`

In most of the cases, the violation is a consequence of long combinational paths introduced by manual replacements of unsupported mathematical functions. Bambu's HLS resource library

does not include functional units for `pow()`, `sqrt()`, `sin()`, `cos()`, or `expf()`. While functionally correct, they produce deep combinational chains that Vivado cannot map within the 10 ns timing budget.

The worst timing violation belongs to `fft_2D_bench` (WNS = -1.064 ns), which required the most extensive rewrite: the entire FFT computation had to be reimplemented from scratch since the original code relied on the external `fftw3` library.

6 Discussion: Implementation Effort and HLS Constraints

Throughout the development of this project, several challenges emerged regarding the suitability of Bambu for complex algorithmic acceleration. This section summarizes the human effort required and discusses whether the conversion to HLS is justifiable for certain kernels.

6.1 Effort Analysis

Table 19 summarizes the modifications required for each benchmark and the estimated effort. The effort is categorized from Low (basic syntax changes) to Critical (complete algorithmic rewrite).

Benchmark	**Modifications Required**	**Effort**	**Complexity**
LRN	Replacement of `pow`	Medium	⋆⋆⋆
Cifar10	Replacement of `pow`, `exp`	High	⋆⋆⋆⋆
Correlation	Replacement of `sqrt`	Medium	⋆⋆⋆
FFT 2D	FFT functions implementation + replacement of `sin/cos`	**Critical**	⋆⋆⋆⋆⋆
FFT (naive)	Replacement of `sin`, `cos`	Low-Medium	⋆⋆
Softmax	Replacement of `exp`	Low-Medium	⋆⋆

Table 19: Summary of modification effort and complexity per benchmark.

The effort estimates in Table 19 are subjective, as they reflect the perspective of the developer carrying out this work.

6.2 Critical Assessment

A key finding of this work is that for kernels heavily dependent on standard mathematical libraries (`math.h`) or specialized libraries like `fftw3`, the use of Bambu (and HLS in general) presents a high entry barrier.

- **Precision Loss:** The reliance on approximations such as Taylor series (e.g., for `sin`, `cos`, `exp`) introduces a precision drift that, while not quantified in this report, could be prohibitive for high-fidelity signal processing.
- **Development Cost vs. Benefit:** Reimplementing basic functions like `pow()` or complex algorithms like FFT from scratch negates one of the primary advantages of HLS: the rapid transition from software to hardware.

Recommendation: The effort invested in adapting these benchmarks was, in most cases, not justified by the results obtained: several designs failed to meet timing constraints or introduced precision loss. When extensive manual rewriting is required prior to synthesis, the primary advantage of HLS, rapid transition from software to hardware, is negated.

7 Conclusions

Overall, the results suggest that Bambu performs well for kernels whose implementation maps directly onto arithmetic operations. Of the 17 benchmarks evaluated, 10 meet the 100 MHz timing constraint under `-DFLOAT`, all of which share a common profile: low resource utilization and logic dominated by multiply-accumulate patterns.

The main limitation observed is Bambu's lack of support for transcendental functions and external libraries. The manual approximations required to work around these constraints introduce deep combinational chains that are responsible for most of the timing violations observed.

Regarding the effect of the datatype configuration, a consistent trend is observed: resource utilization and critical path length both increase monotonically from `-DINT` to `-DFLOAT` to `-DDOUBLE`. Several benchmarks that meet timing under `-DFLOAT` fail under `-DDOUBLE`, confirming that the choice of datatype has a direct and measurable impact on timing closure and should be considered carefully when targeting resource-constrained FPGAs such as the Zynq-7020.

8 Future Work

The next step is to compare the results obtained in this report with those from the MATLAB-based HLS workflow developed in parallel by my classmate, Valentine Ponce, evaluating both tools in terms of resource utilization, operating frequency, and timing closure across the same benchmarks.

If time permits, the same benchmarks will also be synthesized using Vitis HLS, the AMD-native tool integrated with Vivado, which is expected to serve as an optimized baseline for the comparison.

A FPGA Synthesis Metrics

Metric	Description
LUT	Basic configurable logic element. Implements small combinational boolean functions. Reported as *LUT as Logic* (excluding memory-configured LUTs).
LUTRAM	LUTs configured as distributed RAM instead of logic. Used for lightweight on-chip storage.
FF	Flip-Flop. Sequential element holding one bit of state per clock cycle. Used for registers and pipeline stages.
BRAM	Block RAM. Dedicated hard memory primitives for larger on-chip storage needs.
DSP	Digital Signal Processing slice. Hard arithmetic block optimized for multiply-accumulate operations.
Frequency	Operating clock frequency after implementation. Target period is 10 ns (100 MHz) as set by Bambu's TCL script.
WNS	Worst Negative Slack. Timing margin of the critical path. Positive = timing met; negative = timing violation.

Table 20: Description of the FPGA resource and timing metrics reported by Vivado.

B Excluded Datatype Configurations

Table 21 lists the benchmarks for which one or more datatype configurations (`-DINT` or `-DDOUBLE`) were intentionally not evaluated, together with the reason for their exclusion.

Benchmark	Excluded	Reason
`LRN_bench`	`-DINT` `-DDOUBLE`	The algorithm relies on real-valued computations (normalization, exponentiation). Using `-DINT` would cause truncation and loss of precision. The Newton–Raphson approximation uses single-precision literals (`float`), making `-DDOUBLE` functionally equivalent to `-DFLOAT` and therefore not representative.
`cifar_10`	`-DINT` `-DDOUBLE`	The CIFAR-10 pipeline internally calls `lrn` and `softmax`, both of which depend on real-valued arithmetic and single-precision approximations. The same constraints apply.
`cifar_10_multiple`	`-DINT` `-DDOUBLE`	Same as `cifar_10`: the internal functions `lrn` and `softmax` require floating-point arithmetic and use single-precision approximations.
`correlation_2D`	`-DINT` `-DDOUBLE`	The algorithm computes mean values, normalization, and a square root. Using `-DINT` would eliminate all fractional results. The custom `bambu_sqrt` approximation uses single-precision literals, so `-DDOUBLE` would not accurately represent double-precision behavior.
`fft_2D_bench`	`-DINT` `-DDOUBLE`	FFT requires trigonometric twiddle factors and complex-valued rotations, which are incompatible with integer arithmetic. The `bambu_sin` and `bambu_cos` approximations use single-precision literals, making `-DDOUBLE` misleading.
`fft_bench`	`-DINT` `-DDOUBLE`	Same as `fft_2D_bench`: trigonometric twiddle factors preclude integer arithmetic, and the `bambu_sin` approximation uses single-precision literals.
`fft_window_bench`	`-DINT` `-DDOUBLE`	Same as `fft_bench`.
`softmax_bench`	`-DINT` `-DDOUBLE`	Softmax requires exponential evaluation and normalization by division, both of which produce fractional values. The Taylor series approximation uses single-precision literals, making `-DDOUBLE` not representative.

Table 21: Benchmarks excluded from one or more datatype configurations, with justification.

References

[1] PandA Project. *PandA Framework for High-Level Synthesis*. Available at: `https://panda.deib.polimi.it/`

[2] Ferrandi, F. et al. *PandA-Bambu High-Level Synthesis Tool (GitHub Repository)*. Available at: `https://github.com/ferrandi/PandA-bambu`

[3] AMD. *Vivado Design Suite*. Available at: `https://www.amd.com/es/products/software/adaptive-socs-and-fpgas/vivado.html`

[4] OBPMark. *GPU4S_Bench: GPU for Space Benchmark Suite*. Available at: `https://github.com/OBPMark/GPU4S_Bench`